DEEP DIVE

The AI Coding Revolution by the Numbers

Three major reports converge on the same uncomfortable conclusion: AI is transforming how code is written, but engineering organisations are not keeping pace with how code is shipped, secured, or governed.

March 2026 | Engineering Digest | Special Edition

FOREWORD

Within a single month, three of the most rigorous data sources in software engineering published findings that, read together, tell a story none of them quite tells alone. The Pragmatic Engineer surveyed 906 engineers and found AI tool usage at 95% weekly — saturation, not adoption. CircleCI analysed 28 million CI/CD workflows and found that throughput jumped 59% but main branch delivery actually fell. And Laura Tacho's team at DX, drawing on 121,000 developers across 450 companies, measured the productivity gains that everyone claims and found them stuck at roughly 10% — the same plateau they measured when AI coding tools first arrived.

Each report is valuable on its own. Together, they paint a picture that is harder to ignore: the code generation problem is solved. The code delivery problem is getting worse. The productivity gains are real but modest and possibly plateauing. The security implications are largely unaddressed. And the talent pipeline — the pipeline of junior developers who historically learned by doing the work that AI now does — is collapsing.

This deep dive pulls those threads together. It is not a summary of each report but an attempt to understand what they mean in combination, for the people who have to make decisions about engineering teams, tooling investments, and organisational design in 2026.

EXECUTIVE SUMMARY

The State of AI in Software Engineering, March 2026

AI coding tools have reached near-universal adoption. 95% of engineers use them weekly. 75% use them for half or more of their work. Claude Code became the most-loved tool in the market in eight months. This debate is settled.

What is not settled is whether this adoption is producing the outcomes engineering organisations need. CircleCI's analysis of 28 million workflows shows the largest increase in development activity ever recorded — but main branch delivery is *down*, success rates are at a five-year low, and only 1 in 20 teams have figured out how to ship at the speed AI enables. DX's data across 121,000 developers shows productivity gains plateauing at around 10%, with developers saving roughly four hours per week — the same figure as a year ago.

Meanwhile, 45% of AI-generated code contains security vulnerabilities. Trust in AI-generated code is declining even as usage rises. Big Tech junior hiring has fallen from 32% to 7% of new hires. The revolution in how code is written has not been matched by a revolution in how code is reviewed, tested, secured, and shipped. That gap is the defining challenge for engineering leadership in 2026.

THE DATA

The Adoption Story: Near-Universal, Rapidly Consolidating

95%	46%	26.9%	72%
WEEKLY AI ADOPTION	LOVE CLAUDE CODE	AI-AUTHORED CODE	DAILY AI USERS

The Pragmatic Engineer's 2026 survey of 906 engineers — predominantly experienced professionals with a median 11–15 years in the industry — provides the clearest snapshot of where AI tool adoption stands. The headline is unambiguous: 95% use AI tools at least weekly. 75% use them for half or more of their work. 56% report that 70% or more of their work involves AI. These are not early adopter numbers. This is saturation.

The Sonar State of Code survey adds depth: 72% of developers who have tried AI tools now use them daily. 42% of code is currently AI-generated or assisted, up from 6% in 2023, and predicted to exceed half by 2027. This is not a trend line; it's a structural shift in how software gets made.

The tool landscape is consolidating fast

Claude Code launched in May 2025. Eight months later, it is the #1 most-loved AI coding tool at 46% of mentions — 57% when including engineers citing Claude models generally. This happened in a market where GitHub Copilot has 4.7 million paid subscribers and is used by 90% of the Fortune 100. Cursor holds 19% of "most loved" mentions. Copilot sits at 9%.

The enterprise-individual split is striking. At companies with 10,000+ employees, Copilot dominates at 56% — driven by Microsoft procurement bundling and enterprise licensing. At small companies, Claude Code leads at 75%. Engineers choose Claude Code. Procurement departments choose Copilot. Whether those two decisions converge or stay bifurcated will shape the market for the next several years.

Agents are the new frontier

55% of surveyed engineers now use AI agents regularly, up from nearly zero eighteen months ago. Staff+ engineers are the heaviest users at 63.5%. Agent users are twice as excited about AI as non-users. 70% of engineers use two to four AI tools simultaneously. The single-tool era is already over — the question is whether organisations are managing the complexity of multi-tool, multi-agent workflows or simply hoping it works out.

Sources: The Pragmatic Engineer, Sonar State of Code Survey

THE DATA

The Delivery Bottleneck: More Code, Less Shipping

59%	70.8%	72 min	<5%
THROUGHPUT INCREASE	SUCCESS RATE (5YR LOW)	RECOVERY TIME	SHIPPING AT AI SPEED

CircleCI's 2026 State of Software Delivery report is the most important piece of engineering operations data published this year. The analysis covers 28 million CI/CD workflows across thousands of engineering teams. The headline finding — average throughput up 59% year-over-year, the largest increase CircleCI has ever measured — sounds like a triumph for AI-assisted development. It is not.

Feature branches vs. main branch: the divergence

Feature branch throughput for the median team is up 15%. Engineers are writing and pushing more code. But main branch throughput for the same median team is *down* 7%. More code is being written. Less of it is reaching production. The pipeline between "code exists" and "code is deployed" has become the bottleneck, and AI has made it worse by flooding the upstream end with volume.

Even the top 10% of teams show the strain: feature branch throughput up roughly 50%, but main branch throughput up just 1%. The code is getting written. It is not getting shipped.

> *AI is amplifying existing delivery strengths, not distributing them evenly. The top 5% of teams saw throughput increase 97%. The median team got 4%. The bottom quartile got nothing.*
>
> — CircleCI 2026 State of Software Delivery

The success rate collapse

Main branch success rates have fallen to 70.8% — the lowest in over five years. CircleCI's benchmark for a healthy team is 90%. At the current rate, roughly 3 in 10 merges to production are failing. Recovery time has climbed 13% to 72 minutes on average, with mid-sized companies (21–50 employees) averaging three hours.

Put the numbers together at scale: a team processing 500 daily changes at a 70% success rate generates 150 failures per day. At 72 minutes per recovery, that's the equivalent of 12 full-time engineers doing nothing but fixing broken builds. The productivity gains from AI-assisted coding are being consumed — and then some — by the delivery infrastructure's inability to handle the volume and quality of what's being produced.

The 1-in-20 divide

Fewer than 1 in 20 teams have figured out how to ship at AI speed. These top performers share specific characteristics: high test coverage, fast feedback loops, mature CI/CD pipelines, and — critically — engineering cultures that invested in delivery infrastructure *before* AI made code generation faster. Everyone else is producing more work in progress, generating more failed builds, and spending more time on recovery. AI didn't create the delivery bottleneck. It exposed it.

Sources: CircleCI 2026 State of Software Delivery, CircleCI Blog

THE DATA

The Productivity Paradox: The 10% Plateau

~10%	4 hrs	26.9%	50%
PRODUCTIVITY GAIN	SAVED PER WEEK	AI-AUTHORED CODE	FASTER ONBOARDING

Laura Tacho's team at DX surveyed 121,000 developers across 450+ companies — the largest dataset on developer productivity and AI impact available. The finding that should concern every engineering leader making investment decisions based on AI productivity promises: the measured productivity gain is approximately 10%. The same 10% they measured when AI coding tools first took off. The plateau is real.

Developers save roughly four hours per week — the same figure as Q2 2025, after a brief dip to 3.6–3.7 hours in Q4. AI-authored code — defined as code merged with little or no human intervention — now accounts for 26.9% of production code, up from 22% last quarter. More code is being generated with less human involvement, but the overall productivity needle has barely moved.

The self-reporting gap

What developers say

Sonar's survey found developers self-report a 35% personal productivity boost from AI tools. 75% say AI reduces toil. OpenAI's internal data claims developers at OpenAI submit 60% more PRs per week with Codex. These numbers are compelling in isolation.

What the data shows

DX measures approximately 10% actual productivity gain. Actual time spent on toil stays at 23–25% despite 75% of developers saying AI reduces it. 96% don't trust that AI-generated code is functionally correct. 66% spend considerable time correcting "almost-right" AI code. The perception of productivity and the measurement of productivity have diverged.

The METR study: when the control group disappears

The most revealing data point comes not from a survey but from a methodological failure. METR's original study (February–June 2025) found that AI tools caused a 20% *slowdown* among experienced open-source developers — a result so counterintuitive it was widely cited and debated. Their 2026 update attempted to re-examine this finding and ran into a problem that says more than any headline number: they cannot form a control group anymore.

30–50% of developers chose not to submit tasks they were assigned to complete without AI. Recruitment became harder as an increasing share of developers refused to do 50% of their work without AI, even at $50 per hour. The preliminary data suggests speedups (18% for returning participants, 4% for new ones), but the researchers themselves flag the data as unreliable because of selection bias.

> *The inability to form a control group is itself the finding. AI tools have become so embedded in professional practice that measuring their impact through removal has become methodologically impossible. You cannot measure the effect of electricity by asking people to work by candlelight.*
>
> — METR Uplift Study Update, February 2026

The practical implication is this: we may never get clean data on whether AI tools make individual developers faster, because the baseline of "working without AI" no longer exists in a meaningful way. The question for engineering leaders shifts from "does AI make my team more productive?" to

"given that AI is now embedded in how all work gets done, how do I optimise the entire system — including the review, testing, and delivery pipeline that AI doesn't help with?"

Sources: The Pragmatic Engineer / Laura Tacho at Pragmatic Summit, Sonar, METR

THE DATA

The Security Blind Spot: Fast Code, Fragile Code

45%	72%	86%	29%
VULNERABLE AI CODE	JAVA SECURITY FAILURE	XSS FAILURE RATE	TRUST AI ACCURACY

If the delivery bottleneck is the most visible consequence of AI-assisted coding, the security implications are the most dangerous. The numbers are stark: 45% of AI-generated code contains OWASP Top 10 vulnerabilities. Java has a 72% security failure rate. Cross-site scripting (XSS) — one of the most well-understood and preventable vulnerability classes — sees an 86% failure rate across AI models.

These are not edge cases. If 42% of code is now AI-generated (Sonar) and 45% of that contains security vulnerabilities, the maths on your security posture just changed. Roughly one in five lines of production code is AI-generated and potentially insecure. At the rate AI code generation is growing, this ratio worsens every quarter.

Trust is declining, but usage keeps climbing

Stack Overflow's data shows confidence in AI code accuracy fell from 40% to 29% year-over-year. 96% of developers don't fully trust that AI-generated code is functionally correct. 66% spend considerable time correcting "almost-right" AI code — code that looks correct, passes a cursory review, but contains subtle logic errors or security flaws that only surface in production.

The troubling dynamic is that trust and usage are moving in opposite directions. Developers trust AI code less than they did a year ago, but they use it more than ever. This isn't irrational behaviour — it reflects the reality that AI tools are faster, and competitive pressure makes working without them professionally unacceptable. But it means the security review burden is increasing at exactly the moment teams are under pressure to ship faster.

The review illusion

Controlled experiments have found that developers who *felt* 20% faster when using AI tools were actually slower when review time was included. The code generation was faster. The time spent understanding, verifying, debugging, and securing the generated code consumed the gains and then

some. This is the "great toil shift" Sonar identified: old toil (manual coding) replaced by new toil (reviewing, verifying, and managing AI output). The work hasn't disappeared. It's changed shape.

Snyk launched its AI Security Fabric in February 2026 — a continuous defence system across the SDLC — which signals that the industry's security tooling vendors see this as a structural problem, not a temporary one. When security vendors build products specifically for AI-generated code vulnerabilities, the problem has moved from anecdotal to architectural.

Sources: Snyk, Sonar, The Pragmatic Engineer

THE DATA

The Junior Developer Crisis: Who Learns to Code When AI Writes the Code?

32%→7%	73%	50%	26.9%
BIG TECH JUNIOR HIRING	ENTRY-LEVEL ROLES DOWN	FASTER ONBOARDING	CODE MERGED WITHOUT REVIEW

The Pragmatic Engineer's survey contains a data point that received less attention than the adoption numbers but may have longer-lasting consequences: Big Tech junior hiring has plummeted from 32% of new hires in 2019 to just 7% today. Entry-level roles are down 73%. The organisations that historically served as the industry's training ground are dramatically reducing their intake of new engineers.

The conventional explanation is economic: companies are hiring fewer people overall and prioritising experienced engineers who can be productive immediately. But there's a deeper structural shift. When 26.9% of production code is merged with little or no human intervention, the tasks that junior developers traditionally cut their teeth on — writing straightforward CRUD endpoints, fixing simple bugs, building basic features under supervision — are exactly the tasks that AI handles most effectively. The entry-level work is disappearing not because companies don't need it done, but because it no longer requires a human.

The expectation shift

DX's data shows one clear bright spot: onboarding time — measured as time to a developer's tenth pull request — has been cut in half since Q1 2024. New hires are becoming productive faster, which is genuinely valuable. But this metric hides an assumption: the new hires arriving today are expected to be "AI-native" from day one. They're expected to use AI tools to accelerate their ramp-up, to generate code confidently, and to review AI output critically.

This creates a paradox. Junior developers are expected to review and validate AI-generated code — a task that requires the experience and judgement they haven't had the opportunity to develop, because the work that would have built that experience is now done by AI. The career ladder is being restructured while people are still climbing it.

> *The industry is simultaneously reducing the number of junior developers it hires and increasing the level of judgement it expects from the ones who get through the door. These two trends are not compatible over a five-year horizon.*
>
> — Engineering Digest analysis

For engineering leaders, this isn't an abstract talent pipeline concern — it's a structural risk. The senior engineers who currently provide the judgement layer between AI-generated code and production will eventually leave, retire, or move into management. If the pipeline of engineers who develop that judgement through years of hands-on practice has narrowed by 73%, the question is where the next generation of senior engineers comes from. Nobody has a good answer yet.

Sources: The Pragmatic Engineer

IMPLICATIONS

What This Means for Engineering Leadership

The data from these reports converges on five implications that engineering leaders cannot afford to ignore — not because they require immediate action, but because they represent structural shifts in how engineering organisations will operate over the next several years.

Productivity measurement is broken

When developers self-report 35% gains but measured gains sit at 10%, when the METR study can't form a control group, when CircleCI shows 59% more code but 7% less reaching production — the metrics engineering organisations rely on to understand their own performance are no longer trustworthy. Lines of code, PRs merged, velocity points, and even throughput metrics are decoupling from actual business value delivery. The organisations that find new ways to measure what matters — working software in production, not code generated — will have an advantage over those still optimising for output volume.

The delivery pipeline is the bottleneck, not code generation

AI has solved the "writing code" problem for most teams. It has not solved — and is actively worsening — the "shipping code" problem. CircleCI's data is unambiguous: the constraint has

moved downstream. Testing, review, integration, security scanning, and deployment are where AI-generated code piles up and fails. Investment in CI/CD reliability, automated testing, and pipeline performance is no longer an operational concern — it's the primary determinant of whether AI adoption produces value or waste.

Top performers are pulling further ahead

The top 5% of teams saw 97% throughput increases. The bottom quartile got nothing. AI amplifies existing strengths and existing dysfunction equally. Organisations with mature engineering practices — high test coverage, fast feedback loops, clean CI/CD pipelines — are capturing nearly all of the productivity gains from AI tools. Organisations that were already struggling are producing more code, more failures, and more recovery work. The gap is widening, not narrowing.

Security governance is trailing adoption by years

45% vulnerability rates in AI-generated code, combined with 42% of code being AI-generated, creates a compounding security risk that most organisations have not accounted for in their threat models. The security tooling industry is responding (Snyk's AI Security Fabric, OpenAI's Codex Security), but organisational security practices — code review processes, scanning requirements, vulnerability management workflows — were designed for a world where humans wrote all the code. That world no longer exists.

The talent model is shifting beneath everyone's feet

73% fewer entry-level hires. Junior developers expected to be AI-native. Onboarding cut in half but experience requirements climbing. The implicit bargain that defined software engineering careers for two decades — start with simple tasks, build judgement through practice, progress to complex systems work — is breaking down. Whether this produces a generation of engineers who are more capable faster, or a generation who lack the foundational understanding that AI-era senior engineers take for granted, is the most consequential open question in the industry. The answer probably won't be visible for another three to five years, which is exactly why it deserves attention now.

Sources: Analysis based on The Pragmatic Engineer, CircleCI, METR, Sonar, Snyk

OTHER NEWS THIS WEEK

In Brief

Microsoft integrating Claude into Copilot Wave 3 — Anthropic's Cowork technology powering Agent 365, launching May. Claude Opus 4.6 now in Microsoft Foundry. *eWeek, Microsoft Learn*

Revolut secured full UK banking licence — FSCS-protected deposits for 13M customers. Credit products coming. Dual London/NY IPO at $75B valuation being explored. *Bloomberg, CNBC*

PMI and Agile Alliance released Manifesto for Enterprise Agility — first joint manifesto, extending agile principles from team delivery to enterprise governance and funding. *PM Global News*

Pay by Bank live on Amazon and eBay UK — open banking payments hit mainstream e-commerce. eBay partnered with TrueLayer for account-to-account payments via Faster Payments. *Open Banking UK*

March Patch Tuesday: 83 CVEs, 8 critical — notable: Azure MCP Server SSRF (CVE-2026-26118, 8.8 CVSS) and Excel XSS flaw enabling Copilot Agent mode data exfiltration. *Microsoft Security Response Center*

ON THE RADAR

Coming Up

April 19	Claude Haiku 3 model retirement — migrate to Haiku 4.5
April 22	AWS Summit London — expect Bedrock and Trainium announcements
April 29	Zero Day Con, Dublin — Ireland's largest cybersecurity conference
April 30	Node.js 20 end of life
May 1	Microsoft Copilot Wave 3 / Agent 365 + E7 Frontier Suite launch
May 11–12	Dubai Fintech Summit

RECOMMENDED READING

Sources & Further Reading

AI Tooling for Software Engineers in 2026	The Pragmatic Engineer
5 Key Takeaways from the 2026 State of Software Delivery	CircleCI
2026 State of Software Delivery — Full Report	CircleCI
Measuring the Impact of AI on Developer Productivity — Update	METR
State of Code Developer Survey Report	Sonar
AI Security Fabric — Continuous Defence Across the SDLC	Snyk

Engineering Digest

March 2026 · Deep Dive · Special Edition

Next weekly edition: March 20, 2026